

EE 446 Lab 6

Hannah Bevier

Edge impulse link: <https://studio.edgeimpulse.com/public/995616/live>

Part 1

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

The raw inputs are accX, accY, accZ, which are accelerometer readings along the x, y and z axes.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

The features passed into the NN classifier are the spectral features, and there are 33 input features. The names are RMS, Peak 1 Height, Spectral Power 2.0-5.0, Peak 1 Freq, and Peak 2 Freq. All get passed in based on their 3D dimension.

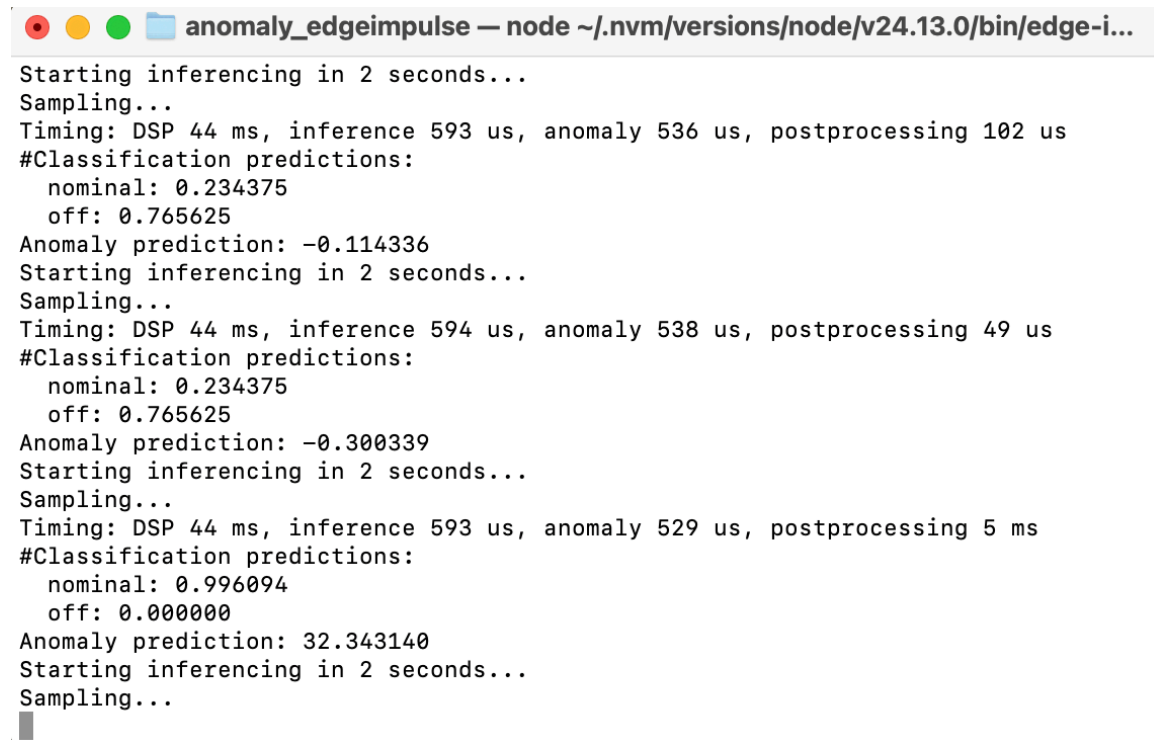
3. NN classifier outputs: State the outputs of the NN classifier.

The outputs are nominal, or off. These are features with those as labels.

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

The features passed into the anomaly detector are the spectral features. The names are accX RMS, accX Peak 1 Height, accY Spectral Power 2.0-5.0, and accZ Spectral power 2.0-5.0/ They are selected out of the features to be included in training the model.

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.



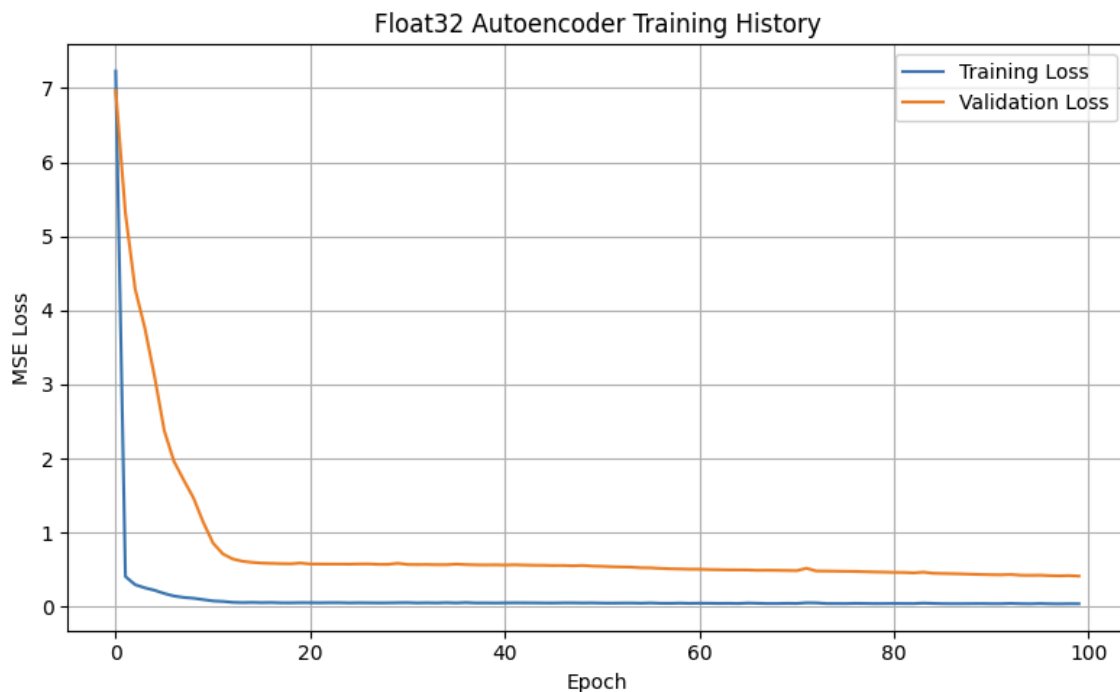
```
anomaly_edgeimpulse — node ~/.nvm/versions/node/v24.13.0/bin/edge-i...
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 536 us, postprocessing 102 us
#Classification predictions:
  nominal: 0.234375
  off: 0.765625
Anomaly prediction: -0.114336
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 594 us, anomaly 538 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.234375
  off: 0.765625
Anomaly prediction: -0.300339
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 593 us, anomaly 529 us, postprocessing 5 ms
#Classification predictions:
  nominal: 0.996094
  off: 0.000000
Anomaly prediction: 32.343140
Starting inferencing in 2 seconds...
Sampling...
```

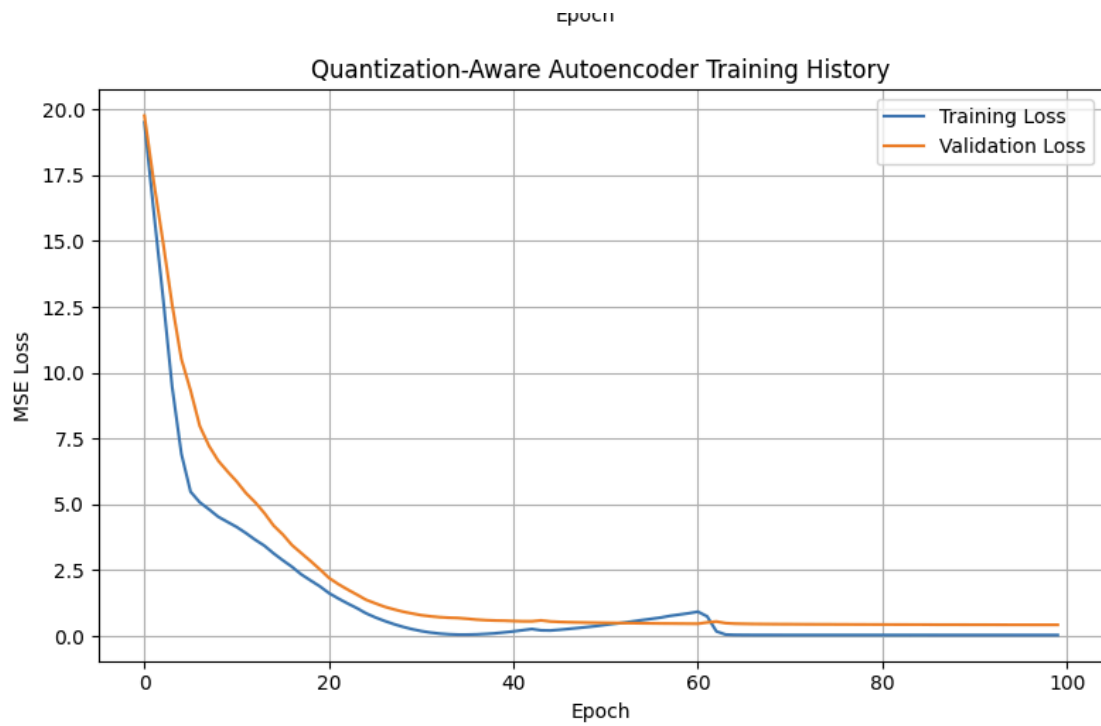
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

When simulating a fan, the anomaly score increased significantly, reaching values close to 1000. When the board was stationary, the anomaly values stayed near zero and were sometimes slightly negative. This indicates that movement caused the input data to differ more from the normal patterns learned during training, resulting in higher anomaly scores. A stationary board closely matched the expected data, so the anomaly score remained low. The classifier output should not always be completely trusted. The anomaly score can saturate at its maximum value, meaning very different levels of motion may still produce the same maximum score. In addition, the reliability of the output depends on how closely the real-world data matches the conditions used during training. Noise, unexpected motion, or conditions not included in the training dataset may reduce accuracy or produce misleading results.

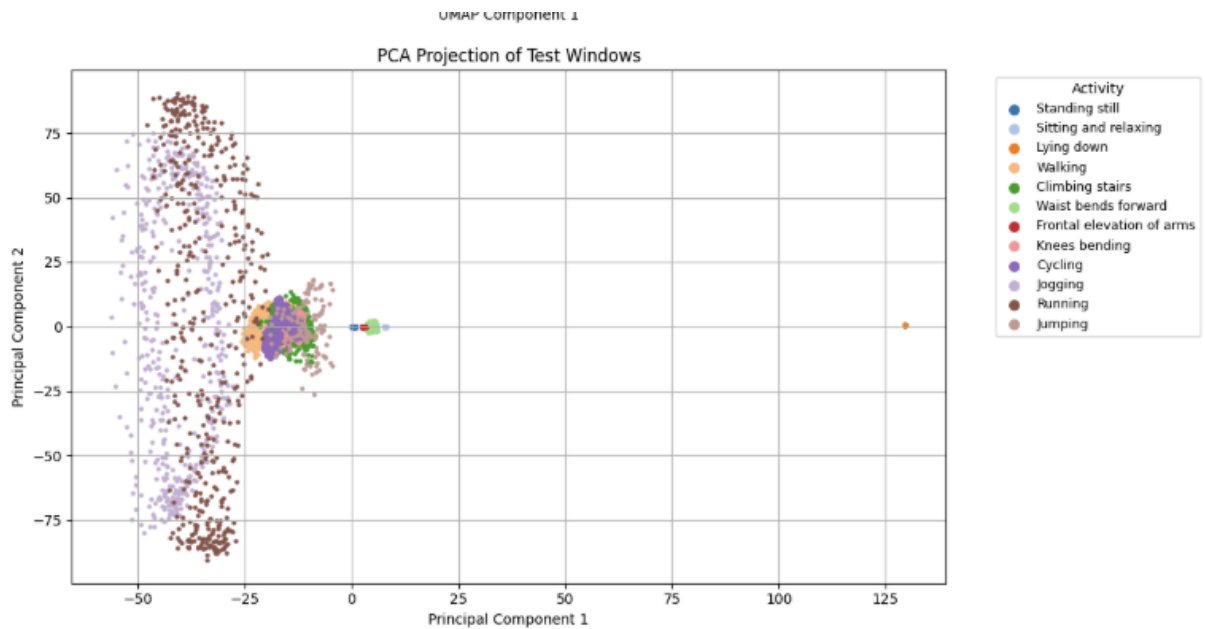
Part 2

- Training and validation loss curves

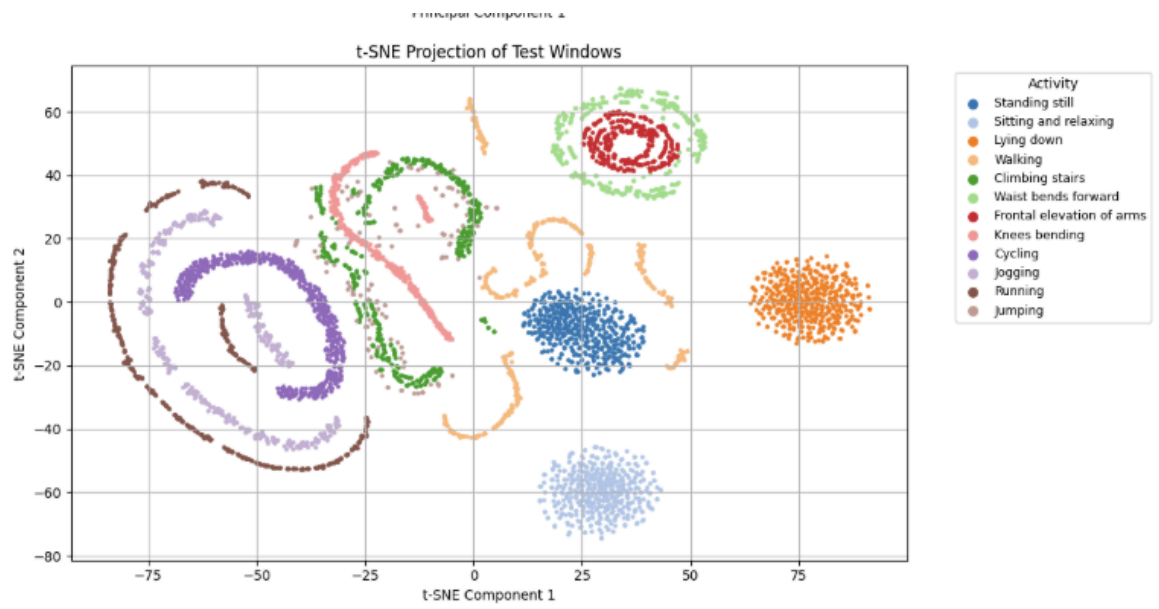




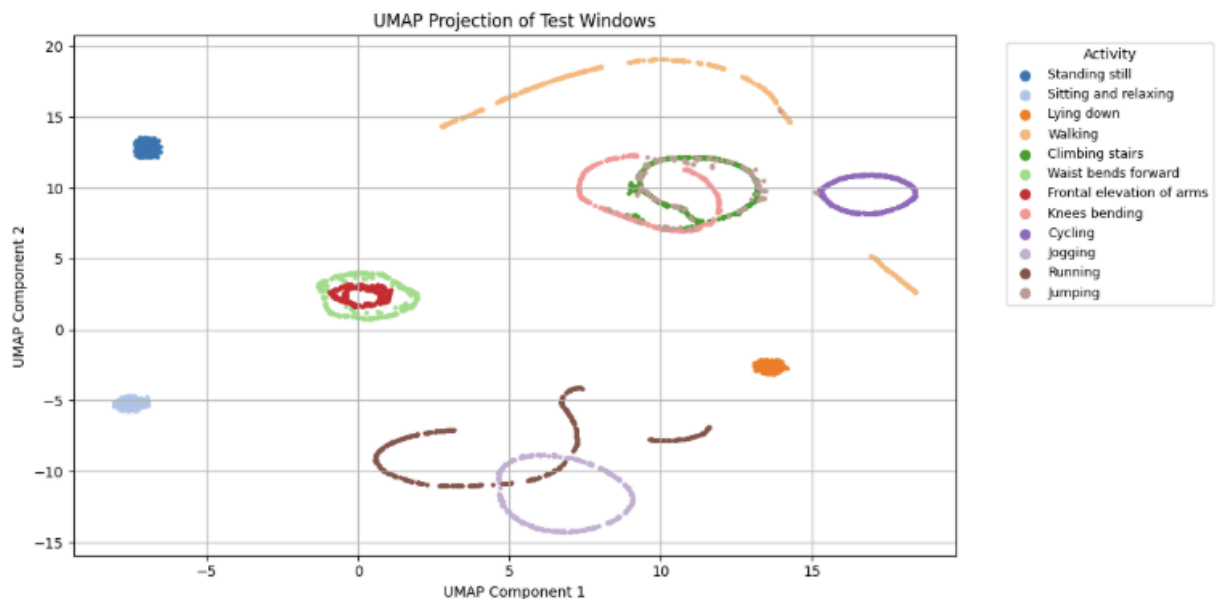
- PCA visualization



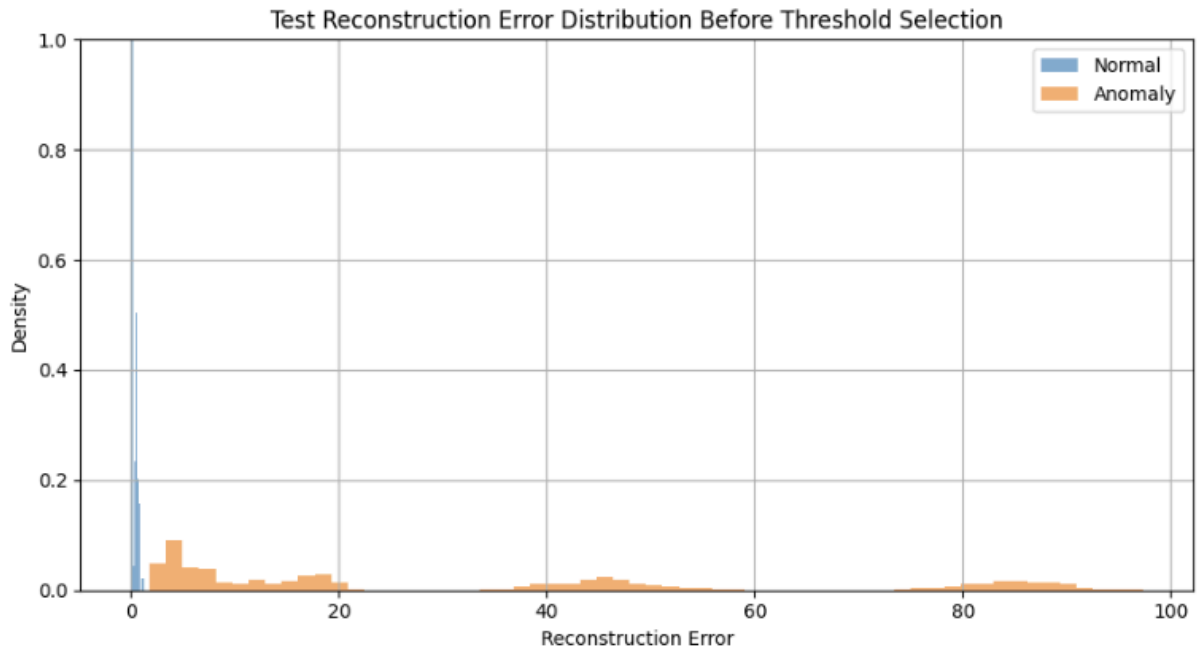
- t-SNE visualization



- UMAP visualization



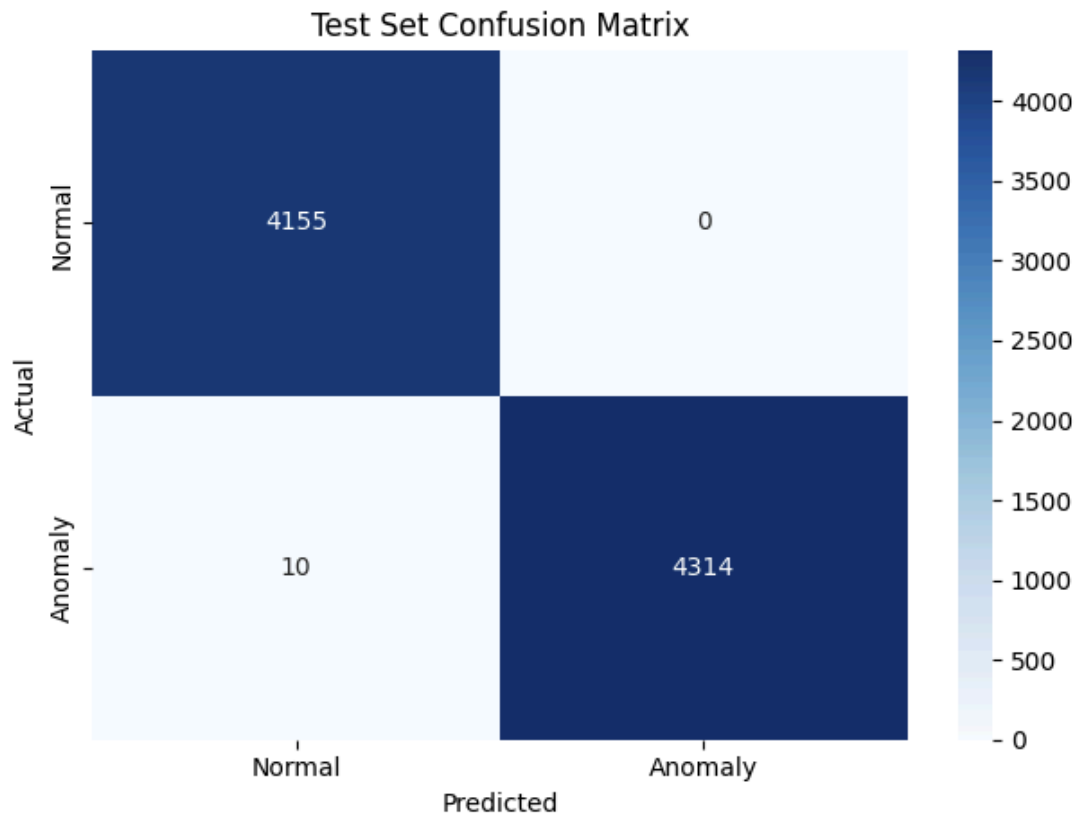
- Reconstruction error distribution



- Selected anomaly threshold



- Confusion matrix



- Classification report or evaluation metrics

Output quantization: (0.086155/126045227, -3)

Train set evaluation: int8 model

	precision	recall	f1-score	support
Normal (0)	0.99	1.00	1.00	10478
Anomaly (1)	1.00	0.99	1.00	10872
accuracy			1.00	21350
macro avg	1.00	1.00	1.00	21350
weighted avg	1.00	1.00	1.00	21350

Test set evaluation: int8 model

	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324
accuracy			1.00	8479
macro avg	1.00	1.00	1.00	8479
weighted avg	1.00	1.00	1.00	8479

Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

```
Result: Normal activity
Reconstruction error: 0.052647
Result: Normal activity
Reconstruction error: 0.052646
Result: Normal activity
Reconstruction error: 0.052653
Result: Normal activity
Reconstruction error: 0.052656
Result: Normal activity
Reconstruction error: 0.052651
Result: Normal activity
Reconstruction error: 0.052657
Result: Normal activity
Reconstruction error: 0.052707
Result: Normal activity
Reconstruction error: 0.052802
Result: Normal activity
Reconstruction error: 0.052694
Result: Normal activity
```

Part 3

- What threshold did you choose for anomaly detection, and why?

I chose 2 as the threshold for anomaly detection, because it is the most balanced. 1 is more sensitive, which causes more false alarms, and 3 is more conservative, so it misses some anomalies.

- How does reconstruction error help distinguish normal and anomalous activity?

Reconstruction error helps distinguish between normal and anomalous activity because the model is trained to accurately reconstruct normal data patterns. When the input matches the patterns learned during training, the reconstruction error is low. However, if the input differs significantly from the learned normal behavior, the model reconstructs it poorly, resulting in a high reconstruction error. Therefore, low reconstruction error indicates normal activity, while high reconstruction error suggests an anomaly.

- What information from the Python notebook must be preserved when deploying the model to Arduino?

The trained model information from the Python notebook must be preserved when deploying the model to Arduino. This includes the model weights, architecture, and quantization parameters used to convert the model from FP32 to INT8. In the notebook, the trained model is converted into a C/C++ source file that contains the quantized model data. This file is then placed in the same project folder as the Arduino sketch so the model can be compiled and uploaded to the Arduino board for inference.

- Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

It is important for the Arduino sketch to use the same preprocessing assumptions as the Python notebook because the model was trained using data processed in that exact way. If the preprocessing steps change, such as scaling, normalization, sampling rate, or feature extraction, the input data presented to the model will differ from the data it learned during training. As a result, the model may incorrectly classify normal inputs as anomalies or produce inaccurate predictions. Consistent preprocessing ensures that the deployed model receives

data in the same format and distribution as the training data, which is necessary for reliable performance.

- What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The main limitation is the limited memory on a microcontroller. The model has to be quantized and undergo a form of memory reduction which downgrades the accuracy. For anomaly detection, some anomalies can go undetected, or there may be some false positives since the accuracy is degraded.